

Sistemas Operacionais

Entrada e Saída - Parte 02

Professor: **Jeremias Moreira Gomes**

E-mail: jeremias.gomes@idp.edu.br

Introdução

Recaptação

- Tipos de Entrada e Saída
- Princípios de Hardware
- Formas de Comunicação

Técnicas de E/S

Princípios de Software para E/S

- Devem possuir **independência de dispositivos**
 - Exemplo: uma instrução read, precisa servir para discos rígidos e pendrives
 - O SO cuida das particularidades
- Nomenclatura uniforme (e independente de software)

Princípios de Software para E/S

- Os objetivos dos princípios de software englobam:
 - **Tratamento de erro**
 - Precisa ser tratado o mais próximo possível do hardware
 - O usuário não deve tomar conhecimento

Modos de Operação de E/S

- Os modos de operação de E/S se diferem de acordo com o tipo de participação da CPU e das interrupções na operação:
 - **E/S programada**
 - **E/S via interrupções**
 - **E/S via acesso direto à memória (DMA)**

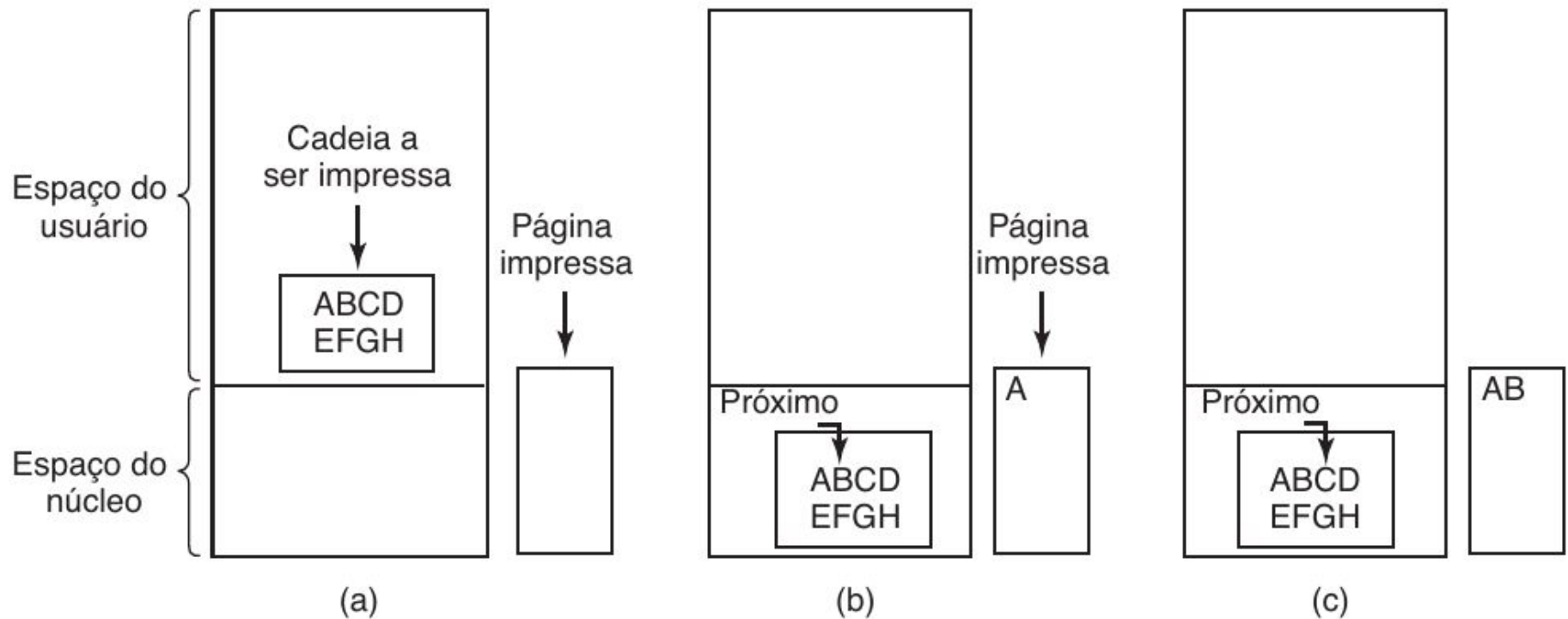
Modos de Operação de E/S - Programada

- Forma mais simples de E/S
 - Tudo é feito pela CPU
- Os dados são trocados entre a CPU e o módulo de E/S

Modos de Operação de E/S - Programada

- A CPU executa um programa que:
 - Verifica o estado do módulo de E/S (ocupado?)
 - Envia o comando de operação
 - Aguarda o resultado
 - Efetua a transferência para registrador da CPU

Modos de Operação de E/S - Programada



Cada operação é controlada pela CPU

Modos de Operação de E/S - Programada

- Desvantagens:
 - CPU fica ocupada o tempo todo
 - Ela realiza toda a E/S
 - Espera ocupada para saber se imprimiu (polling)

Modos de Operação de E/S - Programada

```
copy_from_user(buffer, p, cont);  
for (i=0; i < count; i++) {  
    while (*printer_status_reg !=READY) ;  
    *printer_data_register = p[i];  
}  
return_to_user();
```

```
/* p e o buffer do nucleo */  
/* executa o laço para cada caractere */  
/* executa o laço ate a impressora estar pronta */  
/* envia um caractere para a saida */
```

Modos de Operação de E/S - Interrupções

- CPU requisita um evento (exemplo uma leitura no disco)
- Controladora lê os dados enquanto a CPU faz outras tarefas
 - CPU está liberada para fazer outras coisas
- Controladora envia uma interrupção à CPU
- CPU solicita os dados
- Controladora envia os dados

Modos de Operação de E/S - Interrupções

- Resolve o problema de espera ocupada da CPU
 - CPU ainda participa, mas muito pouco
- CPU:
 - Envia um comando para E/S
 - Executa outra tarefa
 - Envia sinal quando E/S é realizada
 - CPU lê os dados da controladora

Modos de Operação de E/S - Interrupções (exemplo)

IRQ	Uso padrão	Outras utilizações
00	Timer do sistema	Nenhum
01	Teclado	Nenhum
02	IRQs 8 a 15	Modem, placa de vídeo, porta serial (3, 4), IRQ 9
03	Porta serial 2	Modem, placa de som, placa de rede
04	Porta serial 1	Modem, placa de som, placa de rede
05	Placa de som (codec)	LPT2, COM 3 e 4, Modem, placa de rede, HDC
06	FDC	Placa aceleradora de fita
07	Porta paralela 1	COM 3 e 4, Modem, placa de som, placa de rede
08	Relógio de tempo real	Nenhum
09	Placa de som (midi)	Placa de rede, SCSI, PCI
10	Nenhum	Placa de rede, placa de som, SCSI, PCI, IDE 2
11	Placa de vídeo VGA	Placa de rede, placa de som, SCSI, PCI, IDE 3
12	Mouse P/S2	Placa de rede, placa de som, SCSI, PCI, IDE 3, VGA
13	FPU (<i>Float Point Unit</i>)	Nenhum
14	IDE primária	Adaptador SCSI
15	IDE secundária	Placa de Rede, adaptador SCSI

Modos de Operação de E/S - Interrupções

- Interrupções são identificadas por números
- O menor número tem prioridade sobre o maior
- Após uma interrupção, o controlador de interrupções decide o que fazer
 - Envia para a CPU
 - Ignora no momento (dispositivos que lutem)

Modos de Operação de E/S - Interrupções

```
copy_from_user(buffer, p, count);  
enable_interrupts( );  
while (*printer_status_reg != READY) ;  
*printer_data_register = p[0];  
scheduler( );
```

```
if (count == 0) {  
    unblock_user( );  
} else {  
    *printer_data_register = p[i];  
    count = count - 1;  
    i = i + 1;  
}  
acknowledge_interrupt( );  
return_from_interrupt( );
```

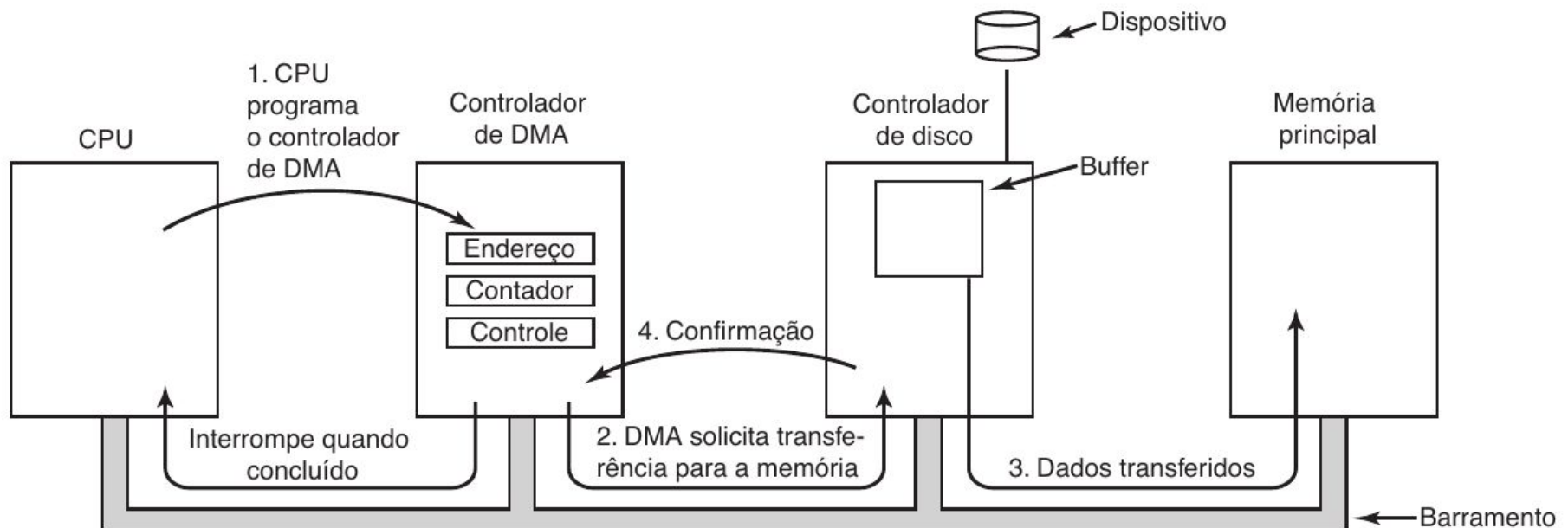
Modos de Operação de E/S - DMA

- Nas outras duas técnicas anteriores:
 - Há um limite na capacidade de transferência da CPU
 - Ela fica ocupada por motivos de gerenciamento
 - Desempenho cai significativamente para muitos dados
- Dessa forma, foi-se desenvolvido uma forma de acesso direto à memória (DMA - Direct Access Memory)

Modos de Operação de E/S - DMA

- O objetivo da DMA é tirar da CPU a função de gerenciamento de E/S
- Agora há uma controladora específica de DMA
 - Esta faz E/S programada, no lugar da CPU

Modos de Operação de E/S - DMA



Modos de Operação de E/S - DMA

- Desvantagens
 - A CPU é mais rápida que a controladora de DMA
 - Se a CPU estiver livre, ela ainda tem que pedir à DMA
- Vantagens
 - E/S programada
 - DMA faz todo o trabalho, liberando a CPU

Princípio de Camadas

Princípio de Camadas

- Facilita a independência dos dispositivos
- Permite modularidade e coesão
- Pode ser dividido entre:
 - **Camadas mais baixas**
 - **Camadas mais altas**

Princípio de Camadas

- **Camadas mais baixas**
 - Detalhes do hardware
 - Drivers e manipuladores de interrupção

Princípio de Camadas

- **Camadas mais altas**
 - Interface para o usuário
 - Aplicações de usuário
 - Chamadas de sistemas
 - Parte independente de entrada e saída

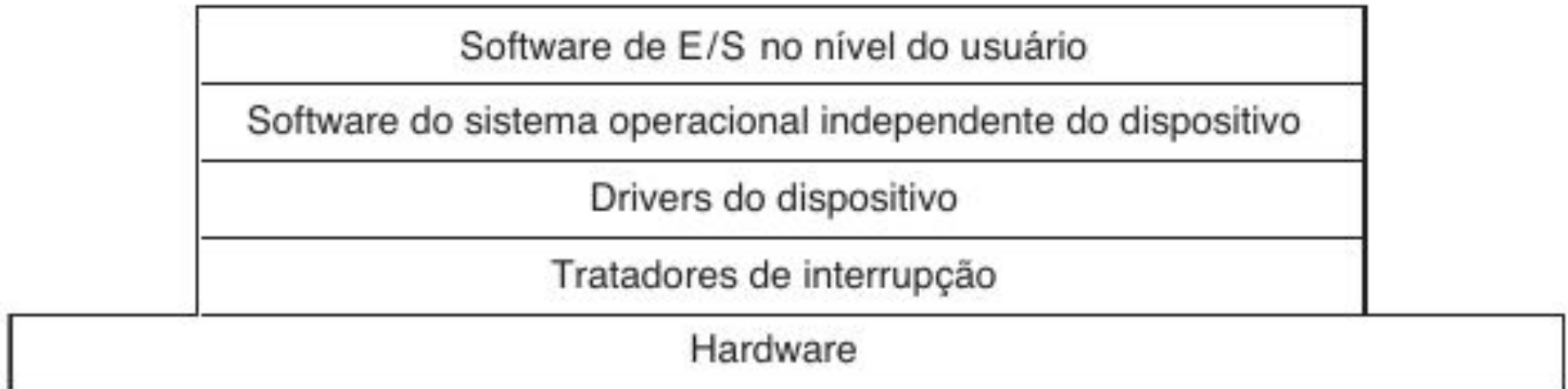
Software de E/S independente de Dispositivo

- Software de E/S:
 - Há partes que são específicas
 - Outras partes são independentes de dispositivos
- Parte Independente
 - Executar E/S comuns a todos os dispositivos

Camadas do Software de E/S

- O software de E/S é normalmente dividido em quatro camadas:
 - **Tratadores de Interrupção**
 - **Software do Sistema Independente do Dispositivo**
 - **Drivers de Dispositivo**
 - **Software de E/S no nível do Usuário**

Camadas do Software de E/S



Tratadores de Interrupção

- Essencial para a E/S como um todo
 - Porém, um pesadelo para programadores
- O ideal é que nunca seja tratado em nenhum ponto a nível de usuário
 - Utiliza semáforos para bloquear operações de E/S

Software de E/S independente de Dispositivo

- Além disso, a ideia do software independente também é fornecer uma **interface uniforme** ao software de usuário
- Evitar que a cada novo dispositivo criado, o SO tenha que ser modificado

Software de E/S independente de Dispositivo

- A parte independente também (1/2):
 - Realiza escalonamento de E/S
 - Provê buffering
 - Ajustes de velocidade e contagem de dados
 - Cache de Dados
 - Armazenar conjuntos de dados frequentes

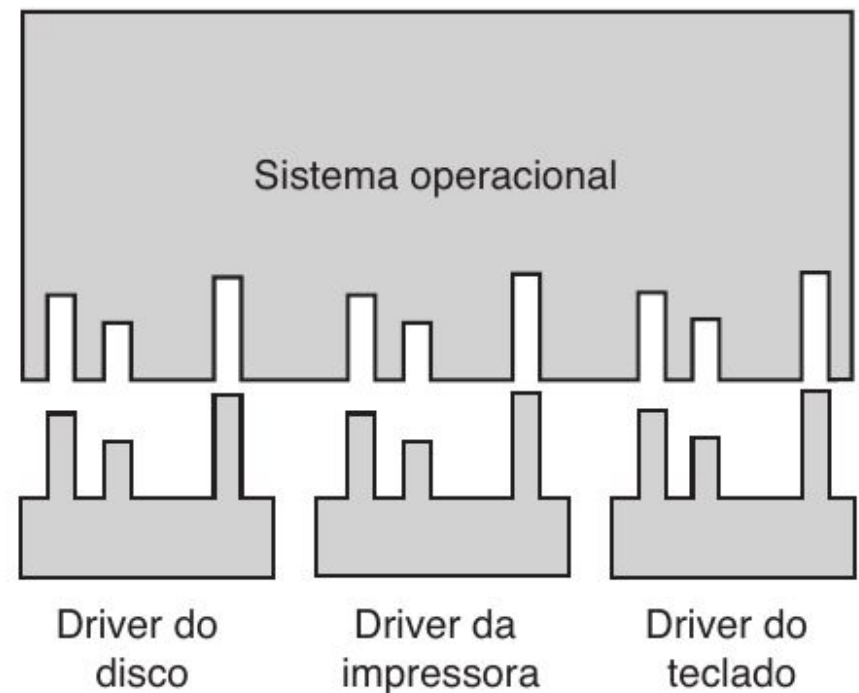
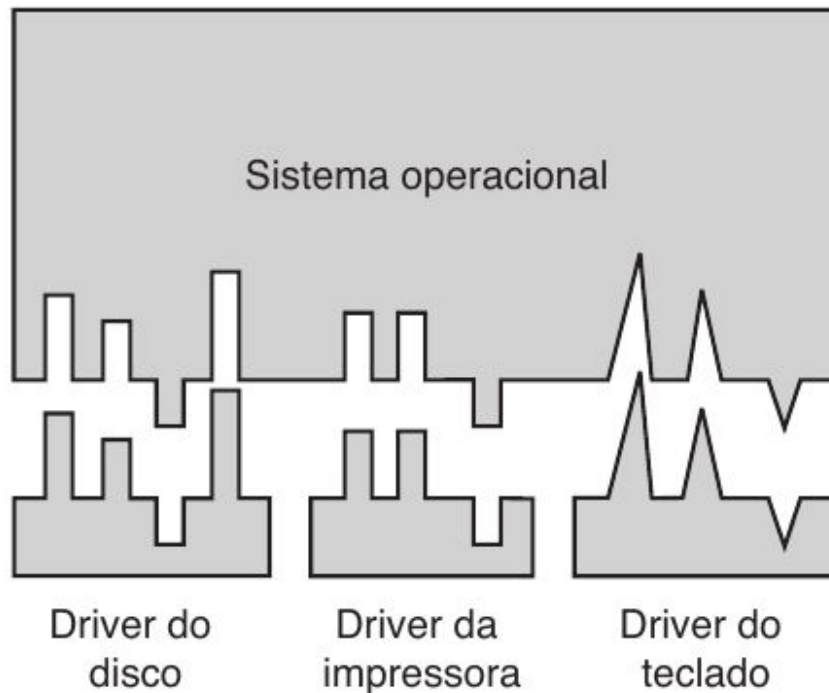
Software de E/S independente de Dispositivo

- A parte independente também (2/2):
 - Reportar erros e proteger contra acessos indevidos
 - Erros de programação, E/S e memória
 - Definir tamanhos de blocos independente de dispositivo

Princípios de Software

- Fornecer interface uniforme
- Cada dispositivo fornece driver com funções
- Padronizar funções fornecidas
- O SO define quais funções o driver irá fornecer

Princípios de Software



Drivers (1/2)

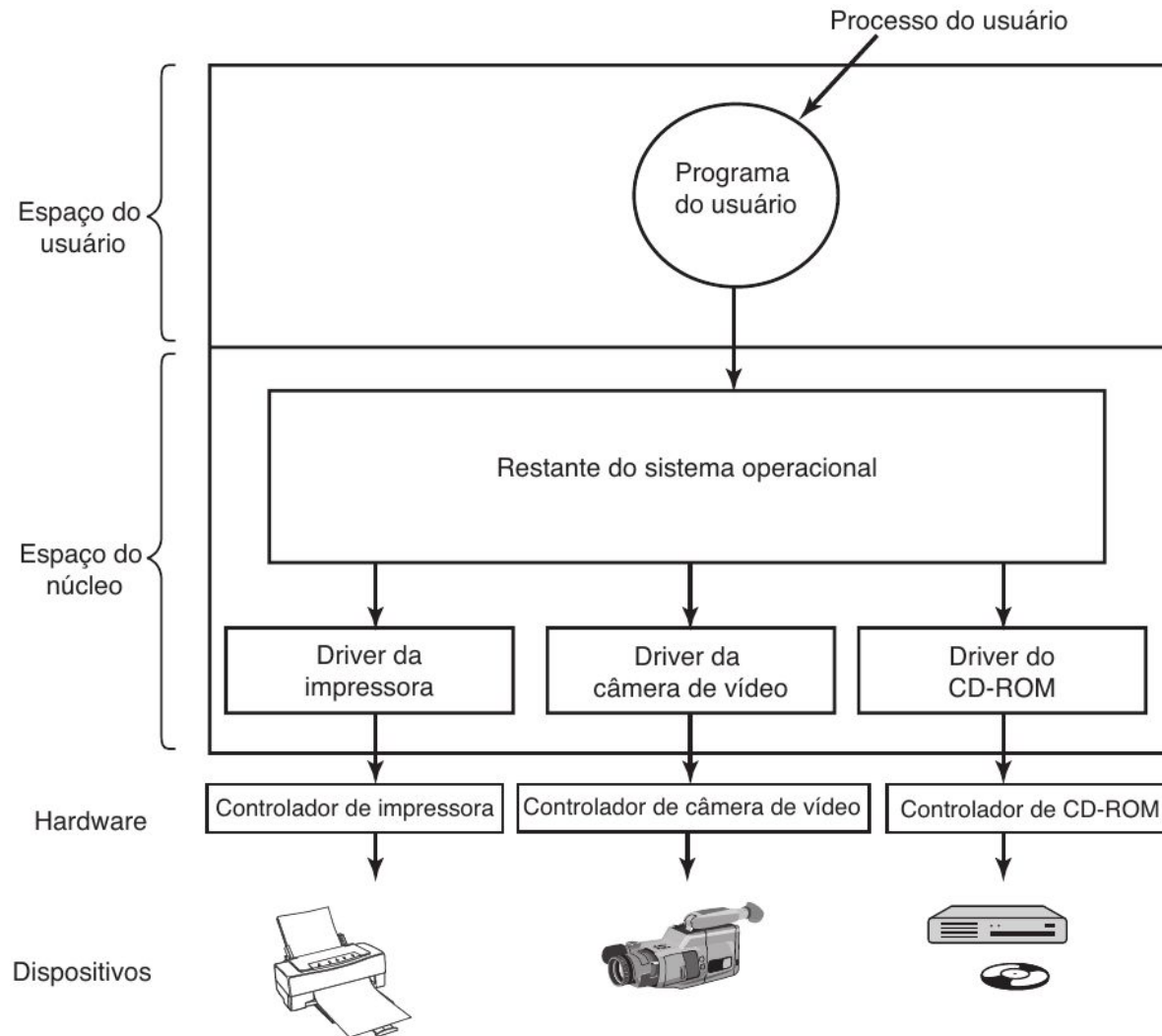
- Particularidades dos dispositivos
 - Registradores
- Código específico para E/S
- Código específico do controlador
- Um código para cada dispositivo
 - Cada versão de um dispositivo tem seu próprio driver

Drivers (2/2)

- Escritos pelo fabricante do dispositivo*
 - Ou por engenharia reversa de versões antigas
 - Cada SO vai ter a sua versão do driver
- Faz parte do Kernel
- Tem acesso privilegiado ao sistema
 - Confiabilidade do sistema

Drivers - Linux

- Utilizando os arquivos de cabeçalho do kernel objetivo
 - Compila o código do driver
 - Insere o driver na lista de módulos
 - Insmod, (lsmod, rmmod, modinfo)
 - Dispositivo pronto para uso



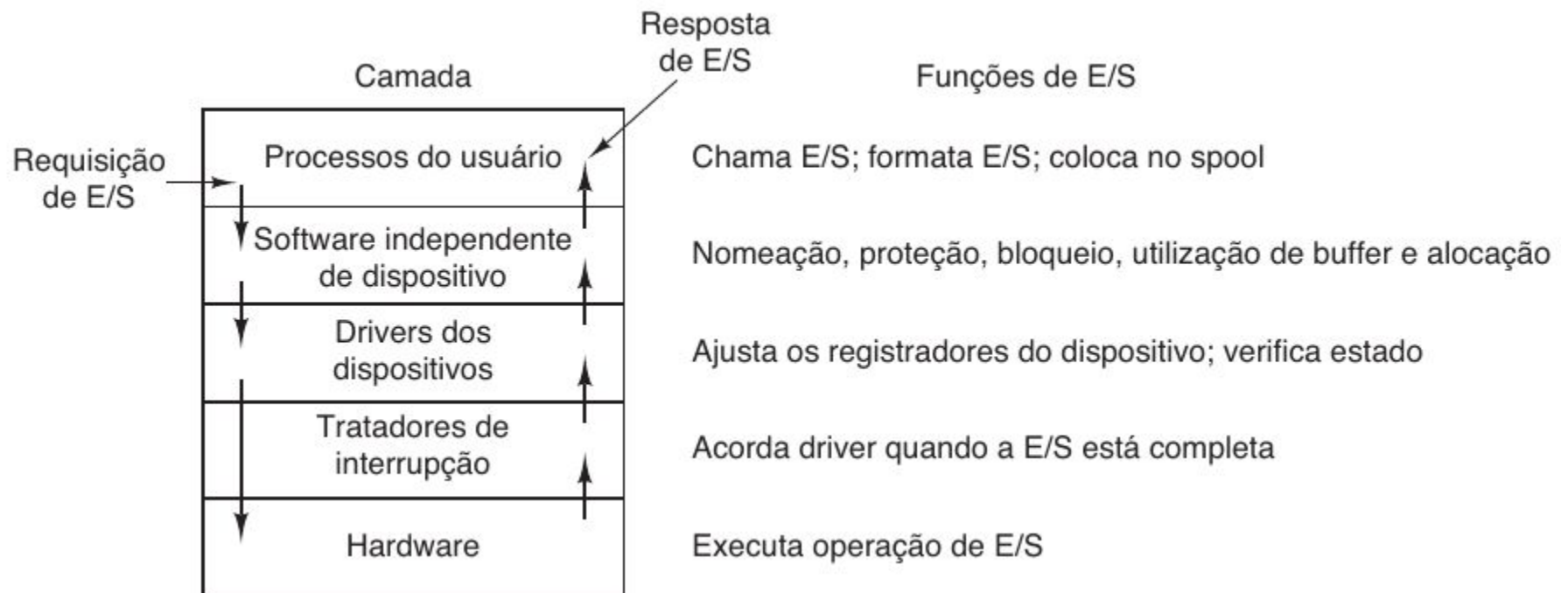
Drivers - Partes do Sistema Operacional

- Definir interface padrão:
 - Drivers de dispositivo de bloco e de caracteres
- Requisitos do SO para seguir a interface
 - Para desenvolvedores de drivers seguirem
- Carregamento dinâmico
 - Objetos compartilhados (linux), DLL (windows), etc

Drivers

- **Exemplos de funções de um driver:**
 - Aceitar pedidos de leitura ou escrita
 - Verificação de operabilidade
 - Inicialização do dispositivo
 - Gerenciamento de energia
 - Registro de eventos (logs)
-

Drivers



Exemplos de Dispositivos de E/S

Relógios

Relógios

- Relógios (também chamados de temporizadores) são elementos essenciais no funcionamento de qualquer sistema multiprogramado
 - Mantém hora
 - Evita monopólio de CPU
 - etc

Relógios

- Relógios podem ter a forma de um driver de dispositivo
 - Dispositivo de bloco
- Podem ser implementados/utilizados:
 - Via hardware
 - Via software

Relógios - Hardware de Relógios

- Relógios simples (de hardware), utilizam a própria rede elétrica com interrupções em ciclos de voltagem
 - 50 a 60 Hz
 - Basicamente não existem mais
- Relógios de oscilador de cristal
 - Mais utilizados nos dias atuais

Relógios - Hardware de Relógios

- **Relógios de oscilador de cristal**
 - Possui três componentes
 - Oscilador de cristal
 - Contador
 - Registrador de apoio

Relógios - Hardware de Relógios

- **Relógios de oscilador de cristal**
 - Gera um sinal periódico de alta precisão
 - É um contador regressivo que dispara uma interrupção
 - Rotina para dar continuidade ao sistema

Relógios - Software de Relógios

- Hardwares de relógio, independente do tipo, são basicamente um contador regressivo
- Todo o restante do funcionamento do sistema enquanto relógio, precisa ser feito via software

Relógios - Software de Relógios

- Funções do software de relógio
 - Manter horário
 - Evitar processos utilizando a CPU mais do que o esperado
 - Tratar chamadas de sistema
 - Fornecer temporizadores

Conclusão